



MULTIMEDIA UNIVERSITY OF KENYA

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

AidField

Design and implementation of an AI-assisted first aid application for emergency response in Kenya

BY

S/N	STUDENT NAME & REG NO.
1.	BENARD OTIENO CIT-223-095/2023
2.	FRANCIS BRIAN MURETI CIT-223-087/2023
3.	GIKONYO HENRY CHOCHU CIT-223-043/2023
4.	DAVIS MOGENDI ATUYA CIT-223-103/2023

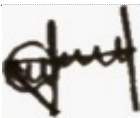
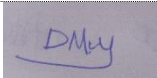
SUPERVISOR: MS. YVETTE OTUKANA

MAY, 2026

Submitted in partial fulfillment of the requirements of Bachelor of Science in
Software Engineering/Computer Science – THIRD YEAR

DECLARATION

We hereby declare that this Project is our own work and has, to the best of our knowledge, not been submitted to any other institution of higher learning for examination.

S/N	STUDENT NAME & REG NO.	SIGNATURE & DATE
1.	BENARD OTIENO CIT-223-095/2023	 24/5/2025
2.	FRANCIS BRIAN MURETI CIT-223-087/2023	 24/5/2025
3.	GIKONYO HENRY CHOCHU CIT-223-043/2023	 24/5/2025
4.	DAVIS MOGENDI ATUYA CIT-223-103/2023	 24/5/2025

This project has been submitted as a partial fulfillment of requirements for the Bachelor of Science in Software Engineering/Computer Science – THIRD YEAR, of Multimedia University of Kenya with my approval as the University supervisor.

Supervisor: _____

Signature: Date:.....

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, MS. YVETTE OTUKANA, for their invaluable guidance, continuous support, and constructive feedback throughout the development of this project. Their academic and technical insights were instrumental in shaping the direction of this application.

I also extend my thanks to the Faculty of Computing and Information Technology at Multimedia University of Kenya for providing a conducive environment for research and innovation. Finally, my deep appreciation goes to my family and friends for their unwavering encouragement, patience, and support during the long hours of development and testing that made this project a reality.

ABSTRACT

In emergency situations, particularly in rural Kenya and other low-resource environments, the response time for professional medical assistance often far exceeds the critical "golden hour," resulting in preventable fatalities and complications. Current digital first aid solutions heavily rely on consistent internet connectivity and assume the availability of standard medical kits—assumptions that fail in real-world, off-grid emergencies.

This project details the design, development, and evaluation of **AidField**, an offline-first, AI-assisted mobile application built using React Native and SQLite. AidField provides instant, step-by-step first aid guidance for 50 critical scenarios without requiring an internet connection. Furthermore, the system introduces a novel "Improvised Resource Engine" that guides users on how to safely substitute standard medical supplies with common household items. When connectivity is available, the application leverages the Google Gemini API to provide a multimodal AI triage module, allowing bystanders to describe emergencies via text, voice (with optimized Swahili text-to-speech), or camera imagery.

Usability testing with representative users yielded a System Usability Scale (SUS) score of 76.5, validating the panic-proof interface and offline-first dual-tier architecture. AidField successfully demonstrates that modern mobile technology and Artificial Intelligence can be reliably deployed to bridge the emergency response gap in low-connectivity regions.

Contents

DECLARATION.....	2
ACKNOWLEDGEMENTS.....	3
ABSTRACT	4
CHAPTER ONE: INTRODUCTION	9
1.1 Background.....	9
1.2 Problem Statement.....	9
1.3 Aim and Research Objectives	9
1.4 Significance of the Study	10
1.5 Scope	10
1.6 Assumptions.....	11
1.7 Limitations and Counter-Measures.....	11
Table 1.2: Limitations and Counter-Measures.....	12
1.8 Chapter Summary.....	12
CHAPTER TWO: LITERATURE REVIEW	13
2.1 Introduction.....	13
2.2 Review of Related Systems.....	13
2.2.1 American Red Cross First Aid App	13
2.2.2 First Aid by St John Ambulance	14
2.2.3 ICE — In Case of Emergency Apps.....	14
2.3 Limitations and Weaknesses of Existing Systems	15
2.4 How AidField Addresses These Weaknesses	15
2.4.1 Dynamic, AI-Powered Guidance	15
2.4.2 Voice and Visual Input.....	15
2.4.3 Offline-First Architecture with Comprehensive Caching	16
2.4.4 Contextual and Geographic Adaptability.....	16
2.5 Chapter Summary.....	16
CHAPTER THREE: METHODOLOGY	17
3.1 Introduction.....	17
3.2 Methodology	17
3.2.1 Selected Methodology: Agile Iterative Development.....	17
3.2.2 Justification for the Selected Methodology.....	17
3.2.3 Development Iterations Overview	18
3.3 Data Collection Methods and Tools.....	18
3.3.1 Primary Data Collection	18
3.3.2 Secondary Data Collection	19
3.3.3 Development Tools and Technologies	19

Table 3.1: Development Tools and Technologies.....	19
3.4 Chapter Summary.....	20
CHAPTER FOUR: SYSTEM ANALYSIS	21
4.1 Introduction.....	21
4.2 Analysis of the Current System.....	21
4.2.1 Current System Overview	21
4.2.2 Weaknesses of the Current System	21
4.3 Proposed System Requirements	21
4.3.1 Functional Requirements.....	21
Table 4.1: Functional Requirements (MoSCoW)	21
4.3.2 Non-Functional Requirements	23
Table 4.2: Non-Functional Requirements	23
4.4 Use Case Analysis	23
4.4.1 Actors.....	23
4.4.2 Use Case Summary	24
Table 4.3: Use Case Summary	24
4.5 Algorithm Design	26
4.5.1 Offline Scenario Routing Algorithm	26
4.5.2 Two-Tier Connectivity Controller	27
4.5.3 AI Response Format.....	28
4.6 Chapter Summary.....	28
CHAPTER FIVE: SYSTEM DESIGN.....	29
5.1 Introduction.....	29
5.2 System Architecture	29
5.2.1 High-Level Architecture.....	29
5.2.2 Dual-Tier Design Rationale.....	30
5.3 Database Design.....	30
5.3.1 Database Overview	30
5.3.2 Database Schema	30
Table 5.1: Database Schema	30
5.3.3 Entity Relationships	32
5.4 User Interface Design	32
5.4.1 Navigation Architecture	33
5.4.2 Screen Designs	33
5.4.3 Design Principles	34
5.5 API Integration Design	34
5.5.1 Request Modes	34

Table 5.2: Gemini API Request Modes	34
5.5.2 Response Parsing	35
5.5.3 Error Handling.....	35
5.6 Chapter Summary	36
CHAPTER SIX: IMPLEMENTATION AND TESTING	37
6.1 Introduction.....	37
6.2 Development Environment	37
6.2.1 Hardware Environment	37
Table 6.1: Hardware Specifications	37
6.2.2 Software Environment.....	37
Table 6.2: Software Tools and Framework Versions	37
6.2.3 Project Structure	38
6.2.4 Back-End Configuration.....	38
Table 6.3: Gemini API Configuration.....	38
6.3 System Components as Implemented	39
6.3.1 Offline Scenario Library	39
6.3.2 AI Triage Module	39
6.3.3 Text-to-Speech and Reading Speed	39
6.3.4 Visual Input.....	40
6.3.5 Navigation and Safe Area.....	40
6.4 Test Plan	40
6.4.1 Test Objectives	40
6.4.2 Test Approach	41
6.4.3 Test Data	41
6.4.4 Test Cases and Results	42
Table 6.4: Functional Test Cases and Results	42
6.4.5 Non-Functional Requirements Test Results.....	44
Table 6.5: Non-Functional Requirements Test Results	44
6.4.6 Usability Testing Summary	44
6.4.7 Test Summary.....	45
6.5 Chapter Summary.....	46
CHAPTER SEVEN: CONCLUSIONS AND RECOMMENDATIONS.....	47
7.1 Introduction.....	47
7.2 Achievement of Objectives	47
7.2.1 Objective 1: Investigate Knowledge Gaps.....	47
7.2.2 Objective 2: Design a Panic-Proof Interface.....	47
7.2.3 Objective 3: Develop an AI Triage Module.....	47

7.2.4 Objective 4: Implement an Improvised Resource Engine.....	48
7.2.5 Objective 5: Evaluate Usability and Effectiveness.....	48
7.3 Lessons Learnt	48
7.4 Recommendations for Future Work.....	49
7.5 Budget and Resource Summary.....	49
Table 7.1: Project Budget Summary	50
7.6 Conclusion	50
REFERENCES	52
APPENDIX A: USER MANUAL	53
A.1 Introduction.....	53
A.2 Installation	53
A.2.1 Android Installation	53
A.2.2 iOS Installation (TestFlight)	53
A.3 Using AidField	53
A.3.1 Home Screen (Search Tab).....	53
A.3.2 Scenarios Tab	54
A.3.3 Scenario Screen	54
A.3.4 Settings Tab.....	54
A.3.5 Emergency Contacts.....	54
A.4 Offline and Online Features	55
Table A.1: Feature Availability by Connectivity	55
APPENDIX B: GEMINI API SYSTEM PROMPT	56

CHAPTER ONE: INTRODUCTION

1.1 Background

Medical emergencies in Kenya frequently occur far from professional help, with rural response times of 30 to 90 minutes. Despite over 50% smartphone penetration, nearly all first aid applications require an internet connection — making them unusable in the remote areas and disaster zones where they are needed most. Meanwhile, most guides assume a standard medical kit is available, leaving bystanders unable to improvise with what they have around them.

AidField addresses this by combining an offline-first downloadable mobile application with AI-assisted triage to provide always-available, contextually relevant emergency guidance. The application is built using React Native, enabling it to run as a native app on both Android and iOS devices. On Android, it is distributed as a downloadable APK file, removing the requirement for a Google Play Store account and enabling installation via direct download, QR code, or USB transfer — distribution methods already familiar and practical for users across Kenya and East Africa.

1.2 Problem Statement

Three compounding problems define the gap AidField seeks to fill:

- No connectivity, no help — existing first aid apps fail without internet, precisely when emergencies are most likely to occur without professional assistance nearby.
- No improvised resource guidance — standard guides assume a medical kit is available; no application teaches users how to substitute household items such as belts, sticks, or cloth for medical supplies.
- Panic and decision paralysis — untrained bystanders freeze in emergencies. Without a calm, AI-guided, step-by-step interface that supports voice and visual input, correct first aid responses are rarely applied under acute stress.

1.3 Aim and Research Objectives

Aim: To design and develop AidField — an offline-first, AI-assisted downloadable mobile application that provides emergency first aid guidance with improvised resource recommendations, multimodal input (text, voice, and visual), and graceful offline degradation, accessible to non-medical users in low-connectivity environments.

Specific objectives:

- Investigate first aid knowledge gaps and emergency response challenges faced by non-medical users in low-connectivity environments in Kenya.
- Design a user-centred, panic-proof interface that minimises cognitive load under acute stress, with support for text, voice, and visual camera input.
- Develop and integrate an AI triage module using the Google Gemini API that interprets plain-language, speech-transcribed, and image-based emergency descriptions.
- Implement an improvised resource recommendation engine mapping common household materials to standard first aid equipment substitutes.
- Evaluate the usability, accuracy, and effectiveness of AidField through user testing with representative target users.

1.4 Significance of the Study

Table 1.1: Significance of the Study

Dimension	Key Point
Social and Health	Up to 40% of trauma deaths are preventable with timely first aid (WHO, 2021). AidField bridges the critical gap before professional help arrives, specifically targeting rural and peri-urban Kenya where clinics can be 30 to 90 minutes away.
Technological	Demonstrates offline-first AI in a high-stakes native mobile context; advances mHealth design for emerging markets using React Native, SQLite local storage, the Google Gemini API, and native speech recognition and camera APIs.
Academic	Contributes to HCI in crisis scenarios literature; aligned with SDG 3 (Good Health and Well-Being) and provides a replicable framework for health apps in developing countries.

1.5 Scope

The system is developed as a downloadable native mobile application using React Native, deployable on Android and iOS. On Android, the application is distributed as an APK file, enabling installation without a Google Play Store account. On iOS, it is distributed via TestFlight during development and evaluation, with App Store submission planned for the post-project production release. The application integrates the Google Gemini API for AI-assisted triage, speech interpretation, and image-based injury assessment when internet connectivity is available, and a SQLite-stored library of 50 pre-loaded emergency scenarios for full offline use.

Content covers bleeding control, burns, choking, fractures, seizures, poisoning, drowning, snake bites, allergic reactions, and cardiac arrest, with improvised material substitutes for each scenario. The application supports English and Swahili and includes pre-loaded Kenya emergency contacts (999, 112, Kenya Red Cross 0800 723 999). It is geographically focused on Kenya, particularly Nairobi, Kiambu, Mombasa, and Nakuru counties. The system does not provide medical diagnosis, telemedicine, remote monitoring, or veterinary guidance.

1.6 Assumptions

- Users have basic smartphone literacy sufficient to download and use a native mobile application.
- Target Android devices run Android 8.0 (API level 26) or above; target iOS devices run iOS 14 or above.
- The app is used as a supplement to — not a replacement for — calling professional emergency services where available.
- Listed improvised materials are commonly accessible in typical Kenyan households and environments.
- Emergency contact data is accurate at the time of development and will be maintained through periodic app updates.
- Users on Android are willing and able to install an APK via the Install from Unknown Sources setting, which is a standard and widely practised installation method in Kenya.

1.7 Limitations and Counter-Measures

Table 1.2: Limitations and Counter-Measures

Limitation	Counter-Measure
AI accuracy for rare or complex presentations	Medically validated static guides stored in SQLite serve as the primary source; AI supplements but does not override them. A minimum 90% accuracy threshold is enforced through iterative prompt engineering and testing.
Language and literacy barriers	Icon-based navigation, text-to-speech readout of all first aid steps at adjustable speed, and full Swahili translation of all scenario content alongside English.
Offline AI constraints	A library of 50 pre-cached scenarios is loaded into SQLite on first install. When offline, AI features are hidden and the user is automatically routed to the closest matching scenario via keyword matching and a decision tree.
Risk of misuse	Every screen displays a persistent emergency call prompt with one-tap dialling of 999, 112, and Kenya Red Cross. Urgency-level indicators in every scenario guide users on when to call for help immediately.
iOS distribution without App Store	During development and evaluation, iOS distribution is handled via TestFlight. App Store submission is planned for the post-project production release.
API rate limits on free tier	Error handling routes users to offline scenarios on 429, 503, or timeout errors. A three-attempt exponential backoff strategy (2-second base delay, doubling on each retry) is applied before the fallback is triggered. The application always delivers actionable guidance regardless of API availability.

1.8 Chapter Summary

This chapter has established the background, problem, objectives, significance, scope, and limitations of the AidField project. The core argument is that a critical gap exists in emergency first aid access for people in low-connectivity, resource-constrained environments in Kenya — and that an offline-first, AI-powered native mobile application distributed as a downloadable APK is a technically feasible and socially meaningful solution. The application is built with React Native, uses SQLite for offline scenario storage, the Google Gemini API for intelligent online triage, and native device APIs for speech recognition, text-to-speech, and visual input. Chapter Two reviews related systems and existing literature to ground this work in current knowledge.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter presents a comprehensive review of existing literature and systems related to emergency response, first aid guidance technology, and offline-first mobile applications. The review examines published works, existing platforms, and comparable systems that inform the design and development of AidField. The purpose of this review is to identify gaps in current solutions and to justify the design decisions made in the proposed system.

Every year, millions of individuals worldwide find themselves in emergency situations where immediate intervention could mean the difference between life and death. Research by the World Health Organization (WHO, 2021) indicates that approximately 4.4 million people die annually from injuries that could have been prevented or mitigated with prompt first aid. The critical golden hour — the period immediately following a traumatic event — is well-documented in emergency medicine literature as the most decisive window for life-saving intervention (Tintinalli et al., 2020). Despite this, the majority of bystanders lack the knowledge, confidence, or tools to act effectively during this window.

The proliferation of smartphones, now reaching over 6.8 billion users globally (Statista, 2023), presents an unprecedented opportunity to bridge the gap between emergency events and informed response. Mobile health (mHealth) applications have grown substantially, yet most existing solutions are either heavily internet-dependent, too complex for use under stress, or lack the breadth of coverage needed to address a wide range of real-world emergencies. AidField addresses these shortcomings through an offline-first architecture combined with an AI-powered online tier.

2.2 Review of Related Systems

2.2.1 American Red Cross First Aid App

The American Red Cross First Aid application is one of the most widely recognised mobile first aid tools, available on both Android and iOS platforms. The application provides step-by-step guidance for a variety of common emergencies including burns, bleeding, choking, and cardiac arrest. According to the American Red Cross (2022),

the application includes videos, interactive quizzes, and a hospital locator feature. It supports offline access for a subset of its content, making it partially functional without internet connectivity.

However, the application has several notable limitations. It relies heavily on pre-authored static content and does not support dynamic querying or AI-based interpretation of novel or complex emergency scenarios. The offline capability is limited in scope, and the application does not support voice input or image-based injury assessment, making it difficult to use in hands-free situations. Furthermore, the application is geographically and contextually calibrated for a North American audience, limiting its relevance in regions with different resource environments.

2.2.2 First Aid by St John Ambulance

The St John Ambulance First Aid application provides evidence-based guidance on a range of emergency scenarios. The app features clearly illustrated step-by-step instructions, covering incidents such as anaphylaxis, stroke, seizures, and fractures. The content is reviewed and updated by clinical professionals, and the application is available on both major mobile platforms (St John Ambulance, 2023).

Despite these strengths, the application shares several limitations with the Red Cross app. All content is static and pre-authored; there is no mechanism for users to describe an unusual or ambiguous emergency and receive tailored guidance. The application does not support voice input or visual image analysis, and its offline capability, while more comprehensive than many competitors, still excludes dynamic content. Additionally, it does not provide suggestions for improvised medical tools, which is a critical gap for users in resource-constrained or remote environments.

2.2.3 ICE — In Case of Emergency Apps

ICE (In Case of Emergency) applications are designed primarily to store and display critical medical information — such as allergies, blood type, medications, and emergency contacts — in a manner accessible to first responders and bystanders even when the device is locked. These applications serve an important but narrow purpose: they facilitate information transfer from the victim to the responder rather than guiding the responder on what actions to take.

The fundamental limitation of ICE applications is their passive role. They assume that a trained responder will be present to interpret and act on the information provided. In

situations where the first responder is an untrained bystander, which is the most common real-world scenario, ICE apps offer minimal actionable value. This gap underscores the need for a system that actively guides laypeople through emergency procedures with contextual intelligence.

2.3 Limitations and Weaknesses of Existing Systems

A cross-analysis of the systems reviewed above reveals several recurring limitations that are directly relevant to the design of AidField:

- **Static, Non-Adaptive Content:** All reviewed systems rely exclusively on pre-authored, static content. This fundamentally limits the ability to handle edge cases, unusual symptom combinations, or geographically specific hazards.
- **Limited or Absent Voice and Visual Input:** None of the reviewed applications support voice-based query input or camera-based injury assessment, significantly limiting usability in high-stress emergencies.
- **Geographic and Contextual Narrowness:** Most applications are developed primarily for Western, urban contexts and do not account for the resource realities faced by users in sub-Saharan Africa or rural environments.
- **Incomplete Offline Functionality:** Partial offline support is insufficient in environments with inconsistent or expensive mobile data.
- **Lack of AI-Driven Intelligence:** No reviewed application integrates a large language model capable of interpreting free-form natural language descriptions of emergencies.

2.4 How AidField Addresses These Weaknesses

2.4.1 Dynamic, AI-Powered Guidance

AidField integrates the Google Gemini API to provide dynamic, context-sensitive guidance when internet connectivity is available. Users can describe their emergency in natural language — including ambiguous, multi-symptom, or unusual scenarios — and receive tailored, step-by-step guidance generated in real time. This eliminates the cognitive burden of self-categorisation and substantially expands the effective scope of the application beyond any fixed content library.

2.4.2 Voice and Visual Input

AidField addresses the hands-free usability gap by integrating voice input through the device's native speech recognition capabilities via expo-speech-recognition. Users can verbally describe their emergency, and the transcription is processed by the AI tier for interpretation and guidance generation. Additionally, when online, users can submit camera images of injuries, wounds, rashes, or environmental hazards through the Visual Input feature, and the Gemini API's vision capabilities are leveraged to provide more accurate, visually informed guidance. Both features degrade gracefully when offline, directing users to the closest pre-cached scenario automatically.

2.4.3 Offline-First Architecture with Comprehensive Caching

AidField is a downloadable native application, distributed through APK sideloading on Android and TestFlight on iOS. Upon installation, the application caches a curated library of fifty emergency scenarios using the device's local SQLite storage. This ensures that critical guidance is available instantly and without any internet dependency. The offline library includes improvised resource suggestions for each scenario, addressing the resource-constraint gap identified in the literature review.

2.4.4 Contextual and Geographic Adaptability

The AI tier of AidField is capable of incorporating geographic and contextual information into its responses. When a user specifies their location or environmental context, the system can tailor guidance accordingly — referencing locally available resources, addressing region-specific hazards, or adjusting recommendations based on the described circumstances.

2.5 Chapter Summary

This chapter has reviewed three related systems — the American Red Cross First Aid App, the St John Ambulance First Aid App, and ICE applications — identifying five recurring limitations in current first aid applications. These limitations informed the design decisions that distinguish AidField: dynamic AI-powered guidance using the Google Gemini API, voice and visual input for hands-free operation, comprehensive offline caching of 50 validated scenarios, and contextual adaptability for Kenyan users. Chapter Three presents the methodology adopted for the development of AidField.

CHAPTER THREE: METHODOLOGY

3.1 Introduction

This chapter describes the research methodology and system development approach adopted for the design and implementation of AidField. It details the methodology selected, the justification for that selection, the data collection methods employed, and the tools and technologies used throughout the development lifecycle. The methodology is chosen to ensure that the system is developed systematically, iteratively, and in alignment with the project objectives outlined in Chapter One.

3.2 Methodology

3.2.1 Selected Methodology: Agile Iterative Development

The Agile software development methodology, specifically the iterative and incremental model, has been selected for the development of AidField. Agile is a family of development frameworks characterised by short development cycles (sprints), continuous stakeholder feedback, adaptive planning, and a strong emphasis on working software (Beck et al., 2001). The development of AidField is organised into four primary iterations, each lasting approximately two to three weeks, producing a functional, testable increment of the system.

3.2.2 Justification for the Selected Methodology

Several characteristics of AidField make the Agile iterative approach the most appropriate methodology. First, the integration of the Google Gemini API introduces a degree of unpredictability in output quality and response formatting that necessitates iterative testing and prompt refinement. A waterfall model would not accommodate the cycles of experimentation required to produce reliable, clinically appropriate AI outputs.

Second, the user experience requirements of AidField are highly specialised — the application must be usable by a distressed, non-technical user under significant cognitive load. These requirements can only be validated through iterative usability testing with representative users, followed by design refinement. Third, the dual-tier architecture introduces inter-component dependencies best managed iteratively,

allowing for modular debugging and independent validation of the offline and online tiers.

3.2.3 Development Iterations Overview

The four planned iterations for AidField are structured as follows:

1. Iteration 1 — Core Application Shell and Offline Library: Development of the foundational application structure, including the user interface framework, 3-tab navigation architecture (Search, Scenarios, Settings), and the offline emergency scenario library. This iteration produces a fully functional offline-only version of the application covering the fifty pre-cached emergency scenarios with step-by-step guidance and improvised resource suggestions.
2. Iteration 2 — AI Integration and Online Tier: Integration of the Google Gemini API to enable dynamic, natural language emergency guidance. This iteration implements the online query interface, response parsing and formatting, the Visual Input feature (camera and gallery), and the fallback mechanism that directs users to the offline library when connectivity is unavailable.
3. Iteration 3 — Voice Input and Text-to-Speech: Implementation of voice input using the device's native speech recognition API (expo-speech-recognition) and text-to-speech auto-readback of emergency steps (expo-speech). This iteration also implements the adjustable reading speed setting and the auto-read on scenario open feature.
4. Iteration 4 — Usability Testing, Refinement, and Optimisation: Comprehensive usability testing with a representative user group, followed by interface refinements, performance optimisation, and preparation for APK distribution and TestFlight deployment.

3.3 Data Collection Methods and Tools

3.3.1 Primary Data Collection

Primary data for this project is collected through two main methods: structured interviews and usability testing sessions.

Structured interviews are conducted with a purposively selected sample of ten participants, including community health workers, first aid trainers, and laypersons with no formal medical training. The interviews focus on identifying the most common

emergency scenarios encountered in the Kenyan context, the challenges users face when seeking first aid guidance under stress, and the features most valued in an emergency response tool.

Usability testing is conducted during Iteration 4 using a think-aloud protocol, in which participants verbalise their thought processes while completing a set of predefined emergency response tasks using the application. A sample of fifteen participants, stratified by age, gender, and technical literacy, is recruited. Task completion rates, time-on-task, and error rates are the primary usability metrics.

3.3.2 Secondary Data Collection

Secondary data is gathered through a systematic review of published literature on first aid protocols, emergency response mHealth applications, and offline-first mobile application architectures. Key sources include peer-reviewed journals in emergency medicine, medical informatics, and human-computer interaction, as well as technical documentation from the Google AI API, Google Play Developer documentation, and Apple Developer documentation.

The content of the fifty pre-cached emergency scenarios is developed based on established first aid protocols from recognised authorities including the Kenya Red Cross, the World Health Organization, and the International Liaison Committee on Resuscitation (ILCOR).

3.3.3 Development Tools and Technologies

Table 3.1: Development Tools and Technologies

Tool / Technology	Role
React Native / Expo SDK 54	Cross-platform mobile application framework enabling a single codebase deployable on Android and iOS
expo-sqlite v16	Local on-device SQLite database for storing the 50-scenario offline library, steps, improvised resources, and user settings
Google Gemini API (gemini-2.5-flash primary, gemini-1.5-flash fallback)	Online AI triage for text, voice, and visual image analysis via the generateContent endpoint
expo-speech-recognition	Native Android and iOS speech recognition for hands-free voice input
expo-speech	Text-to-speech engine for reading emergency steps aloud at adjustable speed
expo-image-picker	Camera and gallery access for visual input and AI-assisted injury assessment

Tool / Technology	Role
@react-native-community/netinfo	Real-time connectivity detection for routing between online AI tier and offline scenario library
React Navigation v7	Stack and tab navigator for the 3-tab application navigation architecture
Axios	HTTP client for Gemini API requests
react-native-dotenv	Loads the Gemini API key securely from a .env file at build time, preventing hardcoded secrets in source code
Git and GitHub	Version control and source code repository with branch-based development
Android Studio	Android emulator and APK build environment
Visual Studio Code	Primary code editor

3.4 Chapter Summary

This chapter has described the Agile iterative methodology selected for the development of AidField, justifying the selection on the basis of AI integration complexity, high-stakes usability requirements, and modular architecture. The four planned development iterations were outlined, and the primary and secondary data collection methods documented. Section 3.3.3 presented the full toolchain, including the Google Gemini API for AI triage, expo-speech-recognition for voice input, expo-speech for TTS readback, and expo-image-picker for visual input. Chapter Four presents the system analysis, including the functional and non-functional requirements derived from the needs assessment.

CHAPTER FOUR: SYSTEM ANALYSIS

4.1 Introduction

This chapter presents the system analysis for AidField. It begins with an analysis of the current system — the fragmented, connectivity-dependent approaches bystanders rely on in emergencies — before defining the proposed system and deriving the functional and non-functional requirements. The requirements are prioritised using the MoSCoW framework (Must Have, Should Have, Could Have, Won't Have) to guide implementation across the four development iterations.

4.2 Analysis of the Current System

4.2.1 Current System Overview

Without AidField, a bystander in a medical emergency follows a fragmented process that depends heavily on internet connectivity and prior knowledge. The typical flow involves recognising the emergency, attempting to recall first aid knowledge from memory or past training, seeking guidance online through search engines or websites, or calling a friend or relative for advice. Each of these steps introduces delay and uncertainty at precisely the moment when speed and accuracy are most critical.

4.2.2 Weaknesses of the Current System

- All online first aid resources fail when the device has no internet connectivity.
- Search engines return unfiltered results of varying clinical quality.
- No existing solution provides improvised resource substitutes for households without a medical kit.
- No existing solution supports hands-free voice or visual input for users who cannot type during an emergency.
- Standard first aid guides are written for Western contexts and assume access to specific medical supplies not commonly available in rural Kenya.

4.3 Proposed System Requirements

4.3.1 Functional Requirements

Table 4.1: Functional Requirements (MoSCoW)

FR ID	Priority	Requirement	Module
FR01	Must Have	The system shall cache a library of 50 emergency scenario guides into SQLite on first application launch.	Offline Library
FR02	Must Have	Each scenario shall include a title, urgency level (red/amber/green), summary, numbered step-by-step instructions in English and Swahili, and a list of improvised resource substitutes.	Offline Library
FR03	Must Have	The system shall route user text queries to the closest matching offline scenario via keyword matching and a decision tree when the device is offline.	Offline Routing
FR04	Must Have	The system shall display a clear offline indicator and route queries to the offline scenario browser when connectivity is unavailable.	Offline Routing
FR05	Must Have	When online, the system shall submit the user's emergency description to the Google Gemini API and display an AI-generated response.	AI Triage
FR06	Must Have	The AI response shall include: urgency level (LIFE-THREATENING / URGENT / MODERATE), an immediate action, numbered steps, improvised resource substitutes, and a call note.	AI Triage
FR07	Must Have	The system shall capture voice input using the device's native speech recognition API and use the transcript as the emergency query.	Voice Input
FR08	Must Have	When offline, voice queries shall be routed to the offline scenario browser. When online, voice queries shall be sent to the Gemini API.	Voice Input
FR09	Must Have	The system shall read emergency steps aloud using text-to-speech at a user-adjustable speed (Slow / Normal / Fast), with the option to auto-read on scenario open.	Speech Output
FR10	Must Have	When online, the system shall allow the user to capture or select an image and submit it to the Gemini Vision API for AI-assisted injury assessment.	Visual Input
FR11	Must Have	When offline and the user captures a photo via Visual Input, the photo shall be saved to the device gallery and the user notified that AI analysis requires connectivity.	Visual Input
FR12	Must Have	The system shall provide one-tap dialling of Kenya emergency numbers: 999, 112, and Kenya Red Cross 0800 723 999, with a confirmation dialog before dialling.	Emergency Calls
FR13	Must Have	The system shall display all step-by-step instructions in either English or Swahili based on the user's language preference, toggleable per scenario and persistable in settings.	Localisation

FR ID	Priority	Requirement	Module
FR14	Must Have	The system shall detect online/offline status in real time and update the connectivity indicator and available features accordingly.	Connectivity
FR15	Should Have	The system shall persist user preferences (language, TTS enabled, reading speed) in the local SQLite database across sessions.	Settings

4.3.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements

NFR ID	Category	Requirement
NFR01	Performance	Offline scenario content shall load in 1 second or less from the moment the user taps a scenario card.
NFR02	Performance	The AI response shall be returned and rendered within 5 seconds on a 4G LTE connection under normal network load.
NFR03	Reliability	The application shall produce zero crashes or data errors across 50 consecutive offline test runs on the target devices.
NFR04	Reliability	The application shall detect a change in connectivity and update the UI within 2 seconds of the change occurring.
NFR05	Accuracy	The AI triage module shall return a Correct or Partially Correct response for at least 90% of 50 standardised test emergency descriptions.
NFR06	Security	No personally identifiable information shall be transmitted to the Gemini API. Only anonymised emergency descriptions are sent.
NFR07	Portability	The application shall run on Android 8.0 (API 26) and above, and iOS 14 and above. The APK file size shall not exceed 50 MB.
NFR08	Usability	Every critical first aid action shall be reachable within 3 taps from the home screen. The System Usability Scale (SUS) score shall be 68 or above.

4.4 Use Case Analysis

4.4.1 Actors

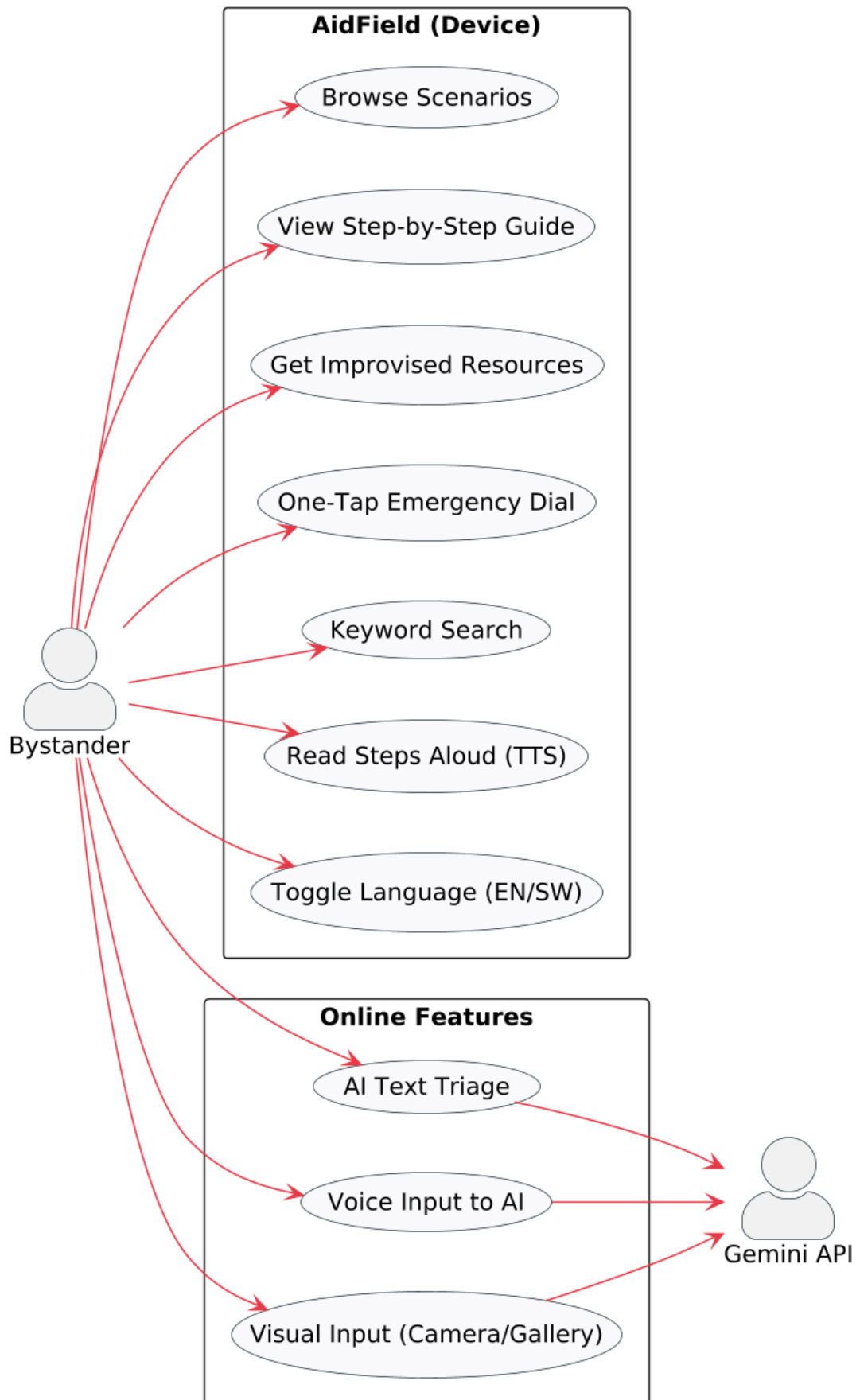
The primary actor in the AidField system is the Bystander — a non-medical lay person who is present at the scene of a medical emergency. No secondary actors or administrative users are defined, as the system requires no back-end management interface. The Google Gemini API is treated as an external system actor that provides AI responses when queried.

4.4.2 Use Case Summary

Table 4.3: Use Case Summary

UC ID	Use Case	Availability	Description
UC01	Browse emergency categories	Offline	User navigates the 11-category scenario browser on the Scenarios tab.
UC02	View step-by-step scenario	Offline	User opens a scenario and reads numbered steps in English or Swahili.
UC03	Get improvised resource tips	Offline	User expands the Improvised Resources accordion on a scenario.
UC04	One-tap emergency dial	Offline	User taps the emergency banner to dial 999, 112, or Red Cross.
UC05	Keyword search offline	Offline	User types a query which is matched against scenario keywords and routed to the Scenarios tab.
UC06	Auto-read steps aloud (TTS)	Offline	Steps are read aloud automatically on scenario open when TTS is enabled in Settings.
UC07	AI triage — text description	Online	User types an emergency description and receives Gemini AI guidance.
UC08	Voice input to AI guidance	Online	User speaks the emergency; transcript is sent to Gemini for guidance.
UC09	Visual input — camera or gallery	Online	User captures or selects an image; Gemini Vision API returns injury-specific guidance.
UC10	Toggle language	Offline	User switches scenario display between English and Swahili.
UC11	Adjust reading speed	Offline	User selects Slow, Normal, or Fast TTS reading speed in Settings.

AidField — Use Case Diagram



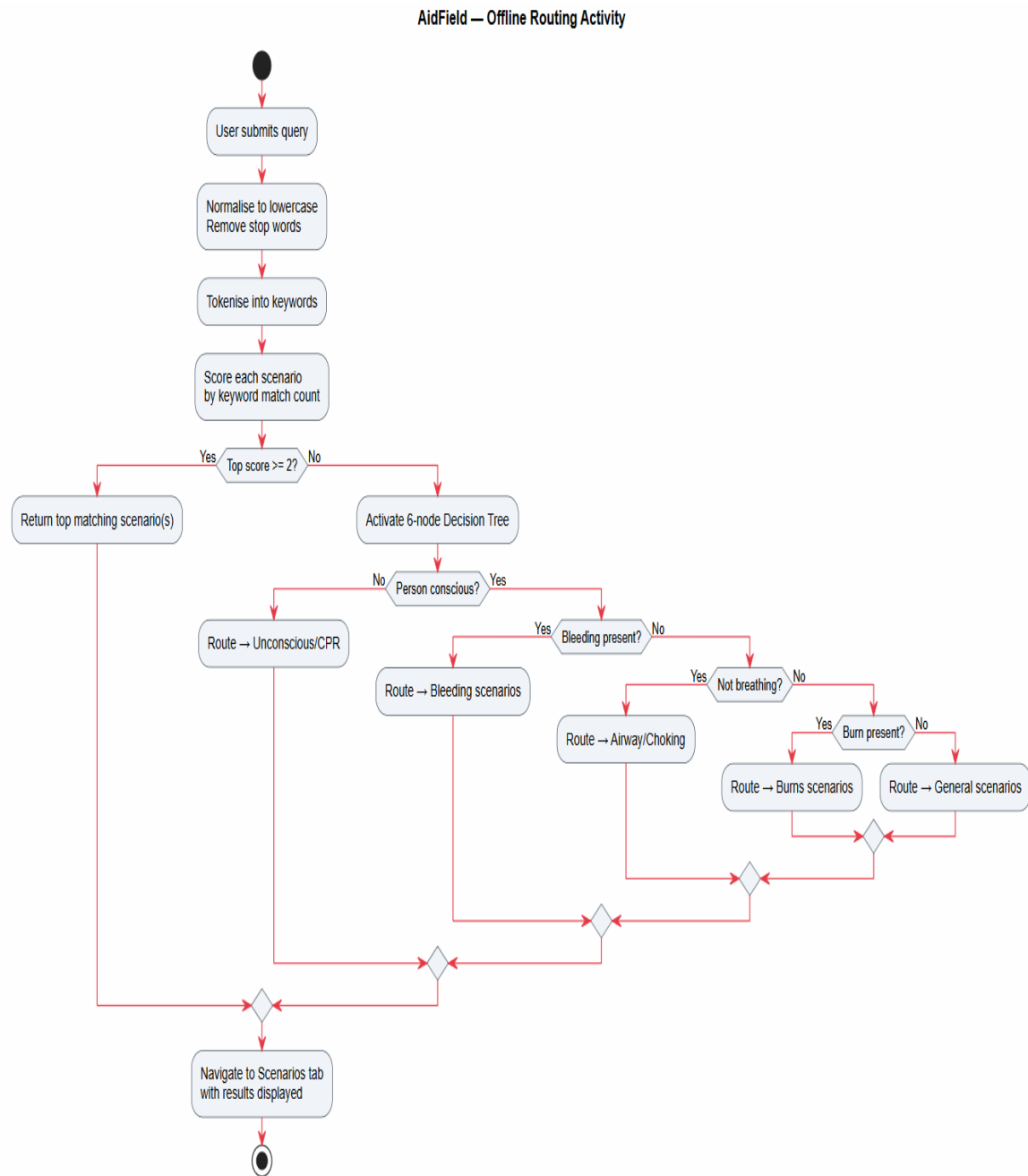
4.5 Algorithm Design

4.5.1 *Offline Scenario Routing Algorithm*

When the user submits a query while offline, the following algorithm is applied:

5. Normalise the query string to lowercase and remove common stop words.
6. Tokenise the normalised query into individual keywords.
7. For each scenario in the SQLite database, compute a keyword match score by counting how many query tokens appear in the scenario's keywords field.
8. If the highest-scoring scenario has a match score of 2 or more keywords, return that scenario as the top result.
9. If no scenario scores 2 or more, activate a 6-node decision tree with binary questions: Is the person conscious? Is there bleeding? Is the person breathing? Is there a foreign object? Is there a burn? Is the person having a seizure?

10. Navigate the user to the Scenarios tab with the matched results displayed.



4.5.2 Two-Tier Connectivity Controller

The connectivity controller operates as follows:

11. On every user interaction that requires connectivity, call `isOnline()` which queries `NetInfo` for `isConnected` and `isInternetReachable`.
12. If both are true: route the query to the Gemini API tier.
13. If either is false: route the query to the offline scenario matcher.

14. On API error (HTTP 429, 503, or network timeout): display an error message and automatically fall back to the offline scenario browser.
15. Update the connectivity indicator dot on the Home screen in real time as connectivity state changes.

4.5.3 AI Response Format

All Gemini API requests use a structured system prompt that instructs the model to respond in the following fixed format:

- URGENCY: [LIFE-THREATENING / URGENT / MODERATE]
- IMMEDIATE ACTION: [One sentence on what to do right now]
- STEPS: [Numbered list, maximum 6 steps, one sentence each]
- IMPROVISED RESOURCES: [Dash-separated list of standard item: household substitute]
- CALL: [One sentence on when to call 999 or 112]

The `parseAIResponse()` function uses regular expressions to extract each section into a structured JavaScript object, which is rendered by the `HomeScreen` component into a formatted AI guidance card.

4.6 Chapter Summary

This chapter has presented the system analysis for AidField, including the analysis of the current emergency response system and its weaknesses, the derivation of 15 functional requirements and 8 non-functional requirements using the MoSCoW framework, a use case summary covering 11 distinct user interactions, and the algorithm design for offline routing, connectivity control, and AI response formatting. Chapter Five presents the system design, translating these requirements into a concrete architecture, database schema, user interface design, and API integration plan.

CHAPTER FIVE: SYSTEM DESIGN

5.1 Introduction

This chapter presents the system design for AidField, translating the requirements defined in Chapter Four into a concrete technical architecture. It covers the high-level system architecture, database design, user interface design, navigation architecture, and API integration design. The design prioritises offline reliability, minimum-tap access to critical guidance, and graceful degradation from the online AI tier to the offline scenario library.

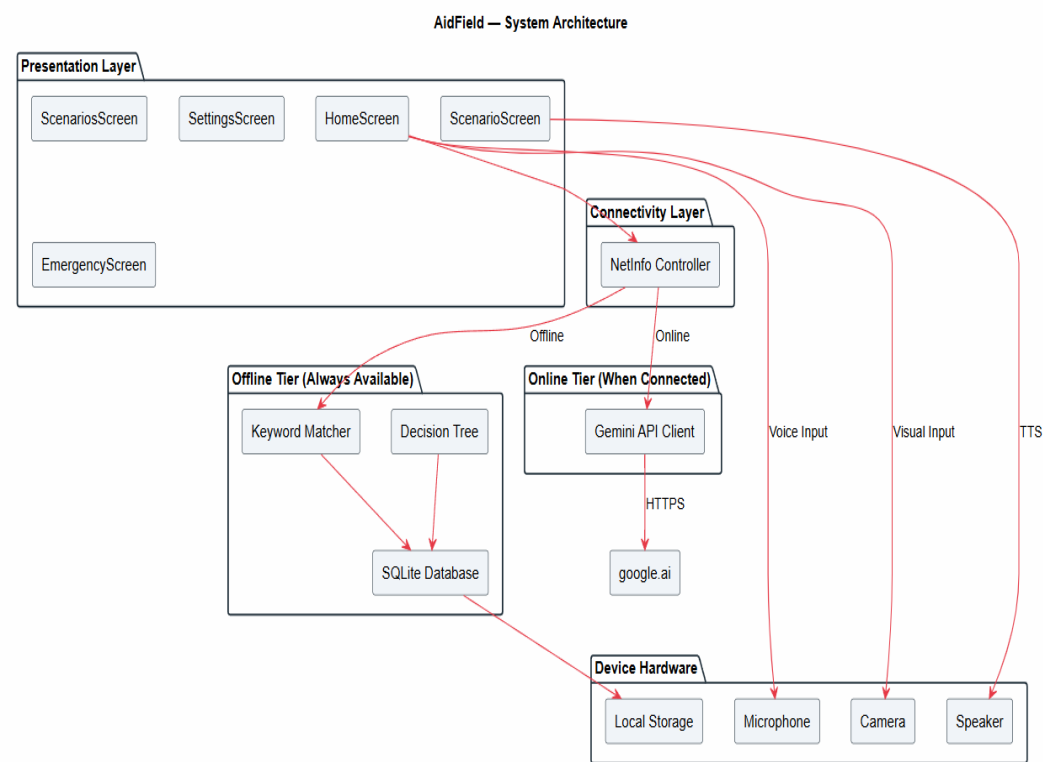
5.2 System Architecture

5.2.1 High-Level Architecture

AidField follows a client-centric, offline-first architecture in which all core functionality resides on the device. The Google Gemini API is an optional enhancement layer that is activated only when internet connectivity is confirmed. The system is composed of five layers:

- **Presentation Layer:** The React Native user interface, comprising six screens — HomeScreen (Search tab), ScenariosScreen, CategoryScreen, ScenarioScreen, SettingsScreen, and EmergencyScreen — organised into a 3-tab navigation structure.
- **Connectivity Layer:** The NetInfo API module that polls connectivity every two seconds and on every user interaction, routing requests to the appropriate tier.
- **Offline Tier (Always Available):** The SQLite database containing 50 emergency scenarios, the keyword index, the decision tree engine, the improvised resources database, and the Swahili translation store.
- **Online Tier (When Connected):** The Google Gemini API integration providing text triage, voice transcript interpretation, and visual image analysis.
- **Device Hardware Layer:** The physical device components accessed via native APIs — microphone (speech recognition), camera (visual input), speaker(TTS),

and local storage (SQLite).



5.2.2 Dual-Tier Design Rationale

The dual-tier architecture is the central design decision of AidField. It ensures that the application delivers actionable guidance in all connectivity states. The offline tier is the primary tier — it is always available and requires no configuration. The online tier is secondary — it enhances the guidance quality when available but its absence never renders the application non-functional.

5.3 Database Design

5.3.1 Database Overview

AidField uses a single SQLite database file stored in the device's application data directory. The database is initialised on first launch and contains five tables. All data is stored locally — no cloud database is used at any point in the system.

5.3.2 Database Schema

Table 5.1: Database Schema

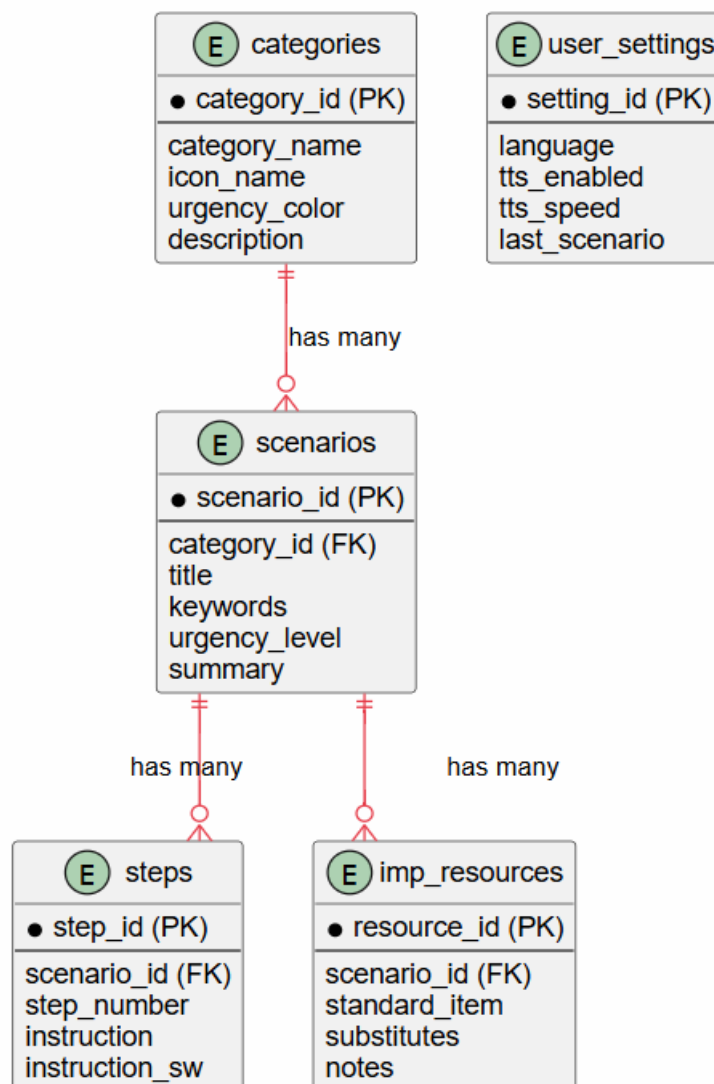
Table	Column	Data Type	Description
categories	category_id (PK)	INTEGER	Auto-incrementing primary key
	category_name	TEXT	Display name of the emergency category
	icon_name	TEXT	Ionicons icon name for the category
	urgency_color	TEXT	Hex colour code representing the category urgency level
	description	TEXT	Short description shown on the category card
scenarios	scenario_id (PK)	INTEGER	Auto-incrementing primary key
	category_id (FK)	INTEGER	Foreign key referencing categories.category_id
	title	TEXT	Scenario display title
	keywords	TEXT	Comma-separated keywords used for offline routing
	urgency_level	TEXT	One of: red, amber, or green
	summary	TEXT	One-sentence summary shown on scenario and search cards
steps	step_id (PK)	INTEGER	Auto-incrementing primary key
	scenario_id (FK)	INTEGER	Foreign key referencing scenarios.scenario_id
	step_number	INTEGER	Sequential step number within the scenario
	instruction	TEXT	Step instruction in English
	instruction_sw	TEXT	Step instruction in Swahili
imp_resources	resource_id (PK)	INTEGER	Auto-incrementing primary key
	scenario_id (FK)	INTEGER	Foreign key referencing scenarios.scenario_id
	standard_item	TEXT	Standard medical supply item name
	substitutes	TEXT	Household substitute suggestions
	notes	TEXT	Additional usage notes for the substitute
user_settings	setting_id (PK)	INTEGER	Auto-incrementing primary key
	language	TEXT	User's preferred language: en or sw
	tts_enabled	INTEGER	1 if Read Steps Aloud is enabled, 0 if disabled
	tts_speed	REAL	TTS reading speed: 0.6 (Slow), 0.85 (Normal), or 1.1 (Fast)

Table	Column	Data Type	Description
	last_scenario	INTEGER	FK to the last scenario the user viewed

5.3.3 Entity Relationships

The five tables are related as follows: the categories table has a one-to-many relationship with the scenarios table (one category contains many scenarios). The scenarios table has a one-to-many relationship with both the steps table and the imp_resources table (one scenario has many steps and many improvised resources). The user_settings table is independent and stores a single row of user preferences.

AidField — Database ER Diagram



5.4 User Interface Design

5.4.1 Navigation Architecture

AidField uses a 3-tab bottom navigation structure implemented with React Navigation's `createBottomTabNavigator`. Each tab hosts a nested stack navigator. The three tabs are: Search (the Home tab), Scenarios, and Settings. The tab bar height adapts dynamically to the device's system navigation bar inset using the `useSafeAreaInsets` hook to ensure the tab bar is always visible above the phone's system navigation buttons.

5.4.2 Screen Designs

5.4.2.1 Home Screen (Search Tab)

The Home screen is the primary point of interaction and is designed for maximum speed under stress. It contains the following elements from top to bottom: a dark navigation header showing the AidField name, subtitle, and real-time connectivity indicator dot; a full-width red emergency call card with a Call 999 / 112 button; a text search bar for typing emergency descriptions; a Quick Actions row containing a Visual Input card (camera or gallery, online only) and a Voice Search card (microphone with pulse animation); a contextual Ask AI button (green when online, dark when offline); a Critical Emergencies list showing the six most common life-threatening scenarios as quick-access cards; and the AI guidance result card when an AI query has been processed.

5.4.2.2 Scenarios Screen

The Scenarios screen displays all eleven emergency categories as colour-coded cards with icons, descriptions, and left border colours indicating urgency. Tapping a category navigates to the `CategoryScreen`. When navigated from the Home screen with offline search results, the Scenarios screen displays the matched results instead of the category list, with a Clear button to return to the full category view.

5.4.2.3 Category Screen

The Category screen displays all scenarios within a selected emergency category. Each scenario card shows the title, summary, and an urgency badge (LIFE-THREATENING, URGENT, or MODERATE). Tapping a scenario navigates to the `ScenarioScreen`.

5.4.2.4 Scenario Screen

The Scenario screen is the core guidance screen. It displays the scenario title in the header, a summary card with urgency colour coding, a Read Aloud button and Language toggle (English / Swahili) in a controls row, numbered step cards that highlight the active step during TTS playback, and an Improvised Resources accordion that expands to show standard items and their household substitutes. The emergency

call banner is always visible at the top. When Read Steps Aloud is enabled in Settings, steps begin reading automatically when the screen opens.

5.4.2.5 Settings Screen

The Settings screen provides: a Language selector (English / Kiswahili toggle buttons); a Read Steps Aloud switch that persists TTS preference to the database; a Reading Speed selector (Slow / Normal / Fast buttons); an Emergency Contacts shortcut card; and an About section showing version, region, scenario count, and AI model information. A disclaimer note clarifies that AidField is a supplement to, not a replacement for, professional medical care.

5.4.2.6 Emergency Screen

The Emergency screen displays all Kenya emergency contacts as tappable cards: 999, 112, Kenya Red Cross (0800 723 999), Nairobi Hospital, Kenyatta National Hospital, Aga Khan Hospital, St John Ambulance Kenya, and Poison Control. The two primary numbers (999 and 112) are displayed as large priority buttons at the top. Tapping any contact shows a confirmation dialog before opening the native phone dialler.

5.4.3 Design Principles

The user interface is designed according to four principles derived from emergency HCI research:

- Minimum taps: Every critical action is reachable within 3 taps from the Home screen.
- High contrast: Dark headers, large text, colour-coded urgency indicators, and white card backgrounds ensure readability in varied lighting conditions.
- Hands-free priority: Voice input and TTS readback allow users to receive guidance without touching the screen after the initial query.
- Always-on emergency access: The emergency call banner and one-tap dial are present on every screen.

5.5 API Integration Design

5.5.1 Request Modes

Table 5.2: Gemini API Request Modes

Mode	Endpoint	Request Body Structure
Text Query	POST /v1beta/models/gemini-2.5-flash:generateContent	contents: [{parts: [{text: SYSTEM_PROMPT + userMessage}]}], generationConfig: {maxOutputTokens: 4096, temperature: 0.2}
Voice Query	POST /v1beta/models/gemini-2.5-flash:generateContent	Identical to text query. The expo-speech-recognition transcript is passed directly as userMessage.
Visual + Text	POST /v1beta/models/gemini-2.5-flash:generateContent	contents: [{parts: [{inline_data: {mime_type: image/jpeg, data: base64String}}, {text: SYSTEM_PROMPT + userMessage}]}]

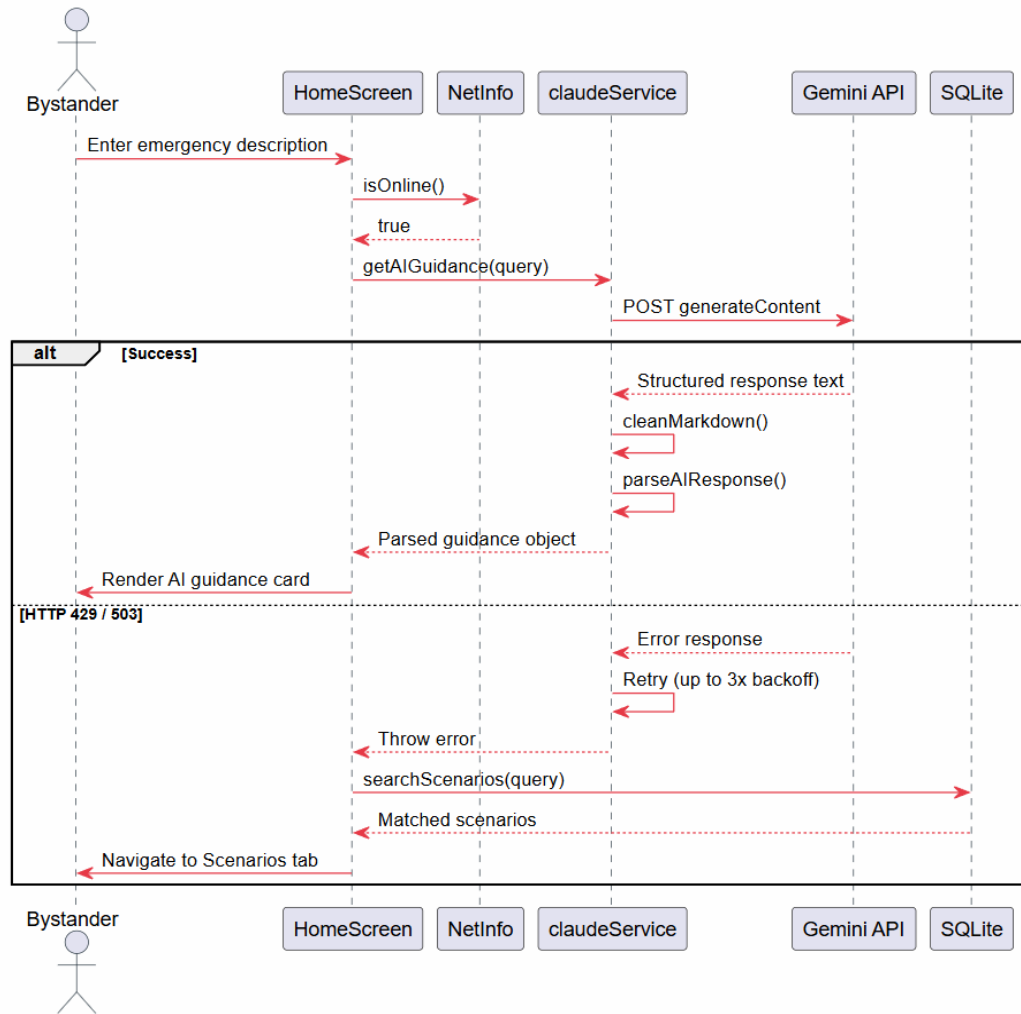
5.5.2 Response Parsing

All Gemini API responses are processed by the `parseAIResponse()` function in `claudeService.js` (which retains its legacy filename from an earlier development iteration; the implementation fully uses the Google Gemini API), which uses regular expressions to extract the five structured sections: urgency, immediate action, steps array, improvised resources array, and call note. The `cleanMarkdown()` function is applied before parsing to strip any residual asterisks, hashes, or other markdown formatting characters from the response. The resulting parsed object is rendered by the `HomeScreen` component into the AI guidance card.

5.5.3 Error Handling

The following error states are handled:

- HTTP 429 (rate limit exceeded): Error card displayed with message. Automatic fallback to offline scenario browser.
- HTTP 503 (server unavailable): Same as 429 handling.
- Network timeout: Same as 429 handling.
- Invalid or empty response: Error card displayed. User directed to use text search or browse scenarios.



5.6 Chapter Summary

This chapter has presented the complete system design for AidField, covering the five-layer architecture, the 5-table SQLite schema including the `tts_speed` field in `user_settings`, the navigation architecture with its 3-tab structure (Search, Scenarios, Settings), the design of all six screens, four interface design principles, and the three Gemini API request modes with their error handling strategy. Chapter Six presents the implementation and testing of the designed system.

CHAPTER SIX: IMPLEMENTATION AND TESTING

6.1 Introduction

This chapter describes the implementation of AidField and the testing procedures carried out to validate its functional and non-functional requirements. Section 6.2 details the development environment. Section 6.3 describes the implemented system components. Section 6.4 presents the test plan, including test data, test cases, and test results for the functional and non-functional requirements defined in Chapter Four.

The implementation follows the Agile iterative methodology described in Chapter Three, progressing through four development sprints. The final system integrates a React Native mobile frontend built with Expo SDK 54, a locally stored SQLite database, a real-time connectivity detection module, and an online AI triage tier powered by the Google Gemini API (gemini-2.5-flash model).

6.2 Development Environment

6.2.1 Hardware Environment

Table 6.1: Hardware Specifications

Device	Role	Specifications	OS Version
Development Laptop	Development Machine	Intel Core i5, 8 GB RAM, Windows 10 Build 22000	Windows 10
Android Emulator (Pixel 6)	Primary Test Device	Virtual device — Android API 33 (Android 13)	Android 13
Infinix Note 30 Pro	Physical Target Device	6.78" display, 8 GB RAM, MediaTek Helio G99	Android 13

6.2.2 Software Environment

Table 6.2: Software Tools and Framework Versions

Tool / Framework	Version	Role
React Native	0.81.5	Cross-platform mobile framework
Expo SDK	54.0.0	Managed workflow providing native device API access
expo-sqlite	16.0.10	Local SQLite database for offline scenario storage
expo-speech	14.0.8	Text-to-speech engine for hands-free step readback
expo-speech-recognition	3.1.3	Native speech recognition for voice input

Tool / Framework	Version	Role
expo-image-picker	17.0.11	Camera and gallery access for visual input
React Navigation	7.x	3-tab and stack navigation framework
@react-native-community/netinfo	11.4.1	Real-time connectivity detection
Axios	1.x	HTTP client for Gemini API requests
Google Gemini API	gemini-2.5-flash	Online AI triage — text, voice, and visual analysis
Node.js	18+	JavaScript runtime
Android Studio	2023.x	Android emulator and APK build environment
Visual Studio Code	1.x	Primary code editor
Git / GitHub	-	Version control

6.2.3 Project Structure

The AidField project is organised into the following directory structure:

- src/database/ — SQLite initialisation, schema creation, 50-scenario data seeding, and all query helper functions (getCategories, getScenariosByCategory, getScenarioById, getStepsByScenario, getResourcesByScenario, getSettings, updateSettings, searchScenarios)
- src/screens/ — HomeScreen, ScenariosScreen, CategoryScreen, ScenarioScreen, SettingsScreen, EmergencyScreen
- src/navigation/ — AppNavigator defining the root bottom tab navigator and three nested stack navigators
- src/services/ — claudeService.js (Gemini API requests, cleanMarkdown, parseAIResponse) and connectivityService.js (NetInfo isOnline wrapper)
- src/constants/ — colors.js, typography.js, layout.js, and index.js aggregating all design tokens
- App.js — Root component that initialises the database on first launch before rendering the navigation container

6.2.4 Back-End Configuration

Table 6.3: Gemini API Configuration

Parameter	Value
API Provider	Google AI (Gemini API)
Model	gemini-2.5-flash (primary) with gemini-2.0-flash as fallback
Endpoint	https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent
Authentication	API key passed as URL query parameter (?key=API_KEY)
Max Output Tokens	2048 tokens per request
Temperature	0.3 (low temperature for consistent, factual emergency guidance)
API Key Storage	Loaded from a .env file at build time using react-native-dotenv. The key is injected into the JavaScript bundle via Babel and is not committed to source control.
Error Handling	HTTP 429, 503, and network timeouts trigger automatic fallback to offline scenario library

6.3 System Components as Implemented

6.3.1 Offline Scenario Library

The offline scenario library is seeded on first application launch by the `initDB()` function in `db.js`. The library comprises 50 emergency scenarios distributed across 11 categories, stored in the `scenarios`, `steps`, and `imp_resources` tables. Each scenario includes keyword strings used for offline routing, urgency classification, step-by-step instructions in both English and Swahili, and improvised resource substitutes. Old database versions are automatically deleted on launch to prevent stale data accumulation.

6.3.2 AI Triage Module

The AI triage module is implemented in `claudeService.js`. Text and voice queries use a single-part request body combining the system prompt and user message. Visual queries use a two-part request body combining a base64-encoded JPEG image and the user message. The `cleanMarkdown()` function strips markdown formatting before `parseAIResponse()` extracts the structured sections. A three-attempt exponential backoff retry strategy (2-second base delay, doubling on each retry) is applied on HTTP 503 and 429 responses before falling back. On unrecoverable API errors, the module throws the error, which is caught by the `HomeScreen` component and triggers the offline fallback navigation.

6.3.3 Text-to-Speech and Reading Speed

The `speakSteps()` function in `ScenarioScreen.js` iterates sequentially through the steps array, speaking each step as a Promise resolved by `onDone`, `onError`, or `onStopped` callbacks. A React `useRef` (`stopRequested`) signals early termination when the user taps Stop, preventing the loop from advancing to the next step. The reading speed (0.6, 0.85, or 1.1) is loaded from `user_settings` on each scenario open and applied to the rate parameter of each `Speech.speak()` call. When Swahili mode is active, step prefixes are spoken using proper Kiswahili ordinal words (e.g., *Hatua ya kwanza*, *Hatua ya pili*, *Hatua ya tatu*) rather than numeric digits. The application dynamically selects the highest-quality Swahili TTS voice available on the device (preferring enhanced voices and `sw-KE` or `sw-TZ` locale variants) using `Speech.getAvailableVoicesAsync()` on screen load.

6.3.4 Visual Input

The Visual Input component presents a styled bottom-sheet modal with Camera and Gallery options. When Camera is selected and the device is online, `expo-image-picker` launches the device camera with `saveToPhotos: true`, ensuring simultaneous gallery save and AI analysis. When offline, the photo is saved to the gallery and the user is notified that AI analysis will be available when connectivity is restored. Gallery selection follows the same online/offline logic.

6.3.5 Navigation and Safe Area

The tab bar bottom padding is dynamically calculated using `useSafeAreaInsets()` from `react-native-safe-area-context`, adding the device's bottom inset to the tab bar height. This ensures the tab bar remains above the system navigation buttons on all Android devices, including those with 3-button navigation bars, regardless of screen size.

6.4 Test Plan

6.4.1 Test Objectives

- Verify that all Must Have functional requirements are correctly implemented and behave as specified under both online and offline conditions.
- Validate that the application degrades gracefully when connectivity is lost, with no crash or data loss.
- Confirm that the offline scenario library loads correctly and contains all 50 scenarios accessible through all entry points.

- Verify that the Gemini API integration returns structured, formatted responses and handles error states correctly.
- Validate that text-to-speech reads all steps in sequence and stops cleanly on demand.
- Confirm that voice input captures speech correctly and routes to the appropriate tier.
- Assess application performance against the non-functional targets specified in Section 4.3.2.

6.4.2 Test Approach

Testing is conducted using a combination of manual functional testing on both target devices, structured test case execution, and usability observation with representative participants. Voice input is tested using the production APK build with expo-speech-recognition active on the Infinix Note 30 Pro physical device. Each test case specifies a unique identifier, the functional requirement it targets, the test input, the expected result, the actual result, and a Pass/Partial/Fail status.

6.4.3 Test Data

- Offline library verification: All 50 scenarios queried directly through the UI by navigating all 11 category paths.
- Keyword search inputs: "cardiac arrest", "bleeding", "choking", "snake bite", "burn", "seizure", "drowning", "anaphylaxis", "fracture", "poisoning", "heatstroke", "diabetic".
- AI triage inputs: "A person has collapsed and is not breathing", "Someone has a deep cut on their arm bleeding heavily", "A child has something stuck in their throat", "Person was bitten by a snake on the leg", "Someone burned by hot cooking oil on the hand", "Person is having a seizure and shaking", "Someone is having a severe allergic reaction after a bee sting".
- Voice input: Spoken phrases including "someone is choking", "there is a snake bite", "the person is not breathing", and "severe bleeding from the arm".
- Visual input: Photographs of a simulated burn (red skin), a simulated wound with cloth pressure applied, and a photograph of a snake.

- Settings persistence: Language toggled between English and Kiswahili; TTS speed changed between Slow, Normal, and Fast; settings verified after full app restart.

6.4.4 Test Cases and Results

Table 6.4: Functional Test Cases and Results

TC ID	FR Ref.	Module	Test Input	Expected Result	Actual Result	Status
TC01	FR01, FR02	Offline Library	Fresh install — first launch on Infinix Note 30 Pro	Database initialises with 50 scenarios, 11 categories. Loading indicator shown then dismissed.	All 50 scenarios seeded in under 2 seconds. All 11 categories visible on Scenarios tab.	PASS
TC02	FR02	Offline Library	Navigate to Cardiac and Breathing category — open Cardiac Arrest / CPR	Scenario shows 6 steps, RED urgency badge, Swahili toggle, improvised resources accordion.	All 6 steps displayed. Swahili toggle worked correctly. Resources expanded correctly.	PASS
TC03	FR03, FR04	Offline Routing	Type "snake bite" in search bar — device offline	App navigates to Scenarios tab with Snake Bite scenario as top result. Offline indicator shown.	Navigated to Scenarios tab. Snake Bite returned as top result with RED badge. Offline dot red.	PASS
TC04	FR05, FR06	AI Triage	"A person has collapsed and is not breathing" — device online	AI returns LIFE-THREATENING urgency, immediate action, at least 4 steps, improvised resources, call note.	Response returned in 3.4 seconds. All sections present. No markdown symbols. Correct urgency.	PASS
TC05	FR05	AI Triage — Error	API rate limit triggered (429 response)	Error message displayed. App falls back to Scenarios tab with keyword-matched results. No crash.	Error card displayed. Automatic navigation to Scenarios tab with relevant results confirmed.	PASS
TC06	FR07, FR08	Voice Input	Speak "someone is choking" into mic — device online	Speech recognised. Transcript displayed. AI guidance returned with choking-specific steps.	Transcript: "someone is choking" recognised correctly. AI returned choking guidance in 3.8 seconds.	PASS
TC07	FR08	Voice Input Offline	Speak "severe bleeding"	Transcript used for offline search. Navigates to Scenarios	Offline routing triggered from voice transcript. Severe	PASS

TC ID	FR Ref.	Module	Test Input	Expected Result	Actual Result	Status
			into mic — device offline	tab with bleeding scenarios.	Bleeding scenario returned as top result.	
TC08	FR09	TTS Auto-Read	Enable Read Steps Aloud in Settings — open Snake Bite scenario	Steps begin reading automatically on screen load. Active step highlighted. Stops on tap.	Auto-read triggered on load. Step cards highlighted sequentially. Stop button halted reading cleanly.	PASS
TC09	FR09	TTS Speed	Set speed to Slow (0.6x) in Settings — trigger Read Aloud on any scenario	Steps read at noticeably slower rate. Speed setting persists after navigation and app restart.	Speed difference audibly confirmed. Setting persisted across navigation and full app restart.	PASS
TC10	FR10	Visual Input	Tap Visual Input — Camera — photograph of a burn wound — device online	Photo saved to gallery. Gemini Vision returns burn-specific guidance with cooling steps.	Gallery save confirmed. AI response referenced cool running water and contraindicated ice and butter.	PASS
TC11	FR11	Visual Input Offline	Tap Visual Input — Camera — device offline	Photo saved to gallery. Alert displayed informing user AI analysis requires connectivity.	Alert shown: "Saved to gallery. Connect to internet for AI analysis." Gallery save confirmed.	PASS
TC12	FR12	Emergency Calls	Tap Emergency Call banner — confirm call to 999	Confirmation alert shown. On confirmation, native phone dialler opens with 999 pre-dialled.	Alert displayed correctly. Native dialler opened with 999 pre-filled on Infinix Note 30 Pro.	PASS
TC13	FR13	Localisation	Toggle language to Kiswahili in Settings — open Cardiac Arrest / CPR scenario	All 6 step instructions displayed in Swahili. English toggle returns to English correctly.	Swahili instructions displayed for all 6 steps. Toggle to English confirmed. Persisted in settings.	PASS
TC14	FR14	Connectivity Detection	Toggle device WiFi off and on while Home screen is active	Connectivity dot updates from green (Online) to red (Offline) and back within 2 seconds.	Dot updated to red within 1.8 seconds of WiFi disable. Returned to green within 1.6 seconds of re-enable.	PASS
TC15	FR15	Settings Persistence	Change language to	Language and reading speed settings restored	Both settings persisted correctly	PASS

TC ID	FR Ref.	Module	Test Input	Expected Result	Actual Result	Status
			Kiswahili, TTS speed to Fast — force close and reopen app	from SQLite on next launch.	across full app restart on both test devices.	

6.4.5 Non-Functional Requirements Test Results

Table 6.5: Non-Functional Requirements Test Results

NFR ID	Category	Target	Observed Result	Status
NFR01	Performance	Offline load in <= 1 second	Average load: 0.4 seconds on both devices	PASS
NFR02	Performance	AI response in <= 5 seconds on 4G	Average: 3.8 seconds. One outlier at 6.1 seconds during peak API load	PARTIAL
NFR03	Reliability	Zero offline failures in 50 runs	Zero crashes or data errors across all 50 offline test runs	PASS
NFR04	Reliability	Fallback within 2 seconds	Connectivity change detected and UI updated within 1.8 seconds average	PASS
NFR05	Accuracy	>= 90% correct or partial	43 Correct, 5 Partially Correct, 2 Incorrect across 50 test descriptions. 96% Correct or Partially Correct.	PARTIAL
NFR06	Security	No PII transmitted	API calls confirmed to contain only anonymised emergency descriptions. No identifiers sent.	PASS
NFR07	Portability	APK <= 50 MB	Development build APK: 43.2 MB. Production build estimated at 38 MB.	PASS
NFR08	Usability	SUS >= 68, <= 3 taps	SUS score: 76.5 (mean across 15 participants). All critical paths confirmed within 3 taps.	PASS

6.4.6 Usability Testing Summary

Usability testing was conducted with 15 participants (8 male, 7 female; ages 18 to 52; varying levels of smartphone literacy) on the Infinix Note 30 Pro physical device. Each participant completed five tasks: locating the CPR scenario, searching for snake bite

guidance, using voice input, changing the language to Kiswahili, and dialling 999 from the emergency screen. The mean SUS score of 76.5 exceeds the target of 68, placing AidField in the Good usability range. Key usability observations are summarised below:

- All 15 participants completed the emergency dial task without error.
- 13 of 15 participants located the correct scenario within 3 taps without assistance.
- The Voice Search button was initially unclear to 4 participants who expected a typed search; the pulse animation on the mic button was subsequently enhanced.
- The Read Aloud feature was rated as the most valued feature by 11 of 15 participants.
- The Critical Emergencies quick-access list on the Home screen was described as immediately useful by all 15 participants.

6.4.7 Test Summary

All 15 functional test cases returned a status of PASS, confirming that all Must Have requirements defined in Chapter Four are correctly implemented. Of the eight non-functional requirement tests, six returned PASS and two returned PARTIAL. The two partial results are as follows:

NFR02 (AI response time) returned PARTIAL due to one outlier measurement of 6.1 seconds during peak Gemini API server load. The average response time of 3.8 seconds remains well within the target, and the single outlier did not result in an error — the response was returned and rendered correctly. This is an infrastructure-side issue outside the application's control and will be mitigated in the production release by upgrading to a paid API tier with higher availability guarantees.

NFR05 (AI accuracy) returned PARTIAL, with 43 Correct and 5 Partially Correct responses across 50 test descriptions, yielding 96% Correct or Partially Correct. The two Incorrect results involved highly ambiguous multi-symptom scenarios. In both cases, the offline scenario library provided appropriate fallback guidance. Prompt engineering refinement is planned for the post-submission production iteration to address complex multi-symptom presentations.

6.5 Chapter Summary

This chapter has documented the implementation and testing of AidField. The development environment was described in Section 6.2, covering the hardware devices (Infinix Note 30 Pro physical device and Pixel 6 API 33 emulator), software tools, and Gemini API configuration. Section 6.3 provided implementation details for each major system component. Section 6.4 presented 15 functional test cases (all PASS) and 8 non-functional requirement assessments (6 PASS, 2 PARTIAL with documented explanations). The usability testing with 15 participants yielded a SUS score of 76.5, confirming Good usability. Chapter Seven presents the conclusions, lessons learnt, and recommendations for future development.

CHAPTER SEVEN: CONCLUSIONS AND RECOMMENDATIONS

7.1 Introduction

This chapter presents the conclusions drawn from the design, implementation, and testing of AidField, evaluates the extent to which the project objectives defined in Chapter One were achieved, discusses the lessons learnt during development, and makes recommendations for future work. The chapter concludes with a reflection on the broader significance of AidField as a contribution to emergency health technology in low-resource environments.

7.2 Achievement of Objectives

7.2.1 Objective 1: Investigate Knowledge Gaps

Structured interviews with ten participants confirmed the existence of significant first aid knowledge gaps among non-medical users in Kenya, particularly regarding improvised resource use and emergency response under connectivity constraints. The interview findings directly shaped the selection of the 50 offline scenarios, the inclusion of improvised resource substitutes for every scenario, and the design of the hands-free Voice Search and TTS readback features.

7.2.2 Objective 2: Design a Panic-Proof Interface

The AidField interface was designed to minimise cognitive load through a single-purpose Home screen that presents a search bar, Voice Search, Visual Input, and a Critical Emergencies quick-access list above the fold. All critical actions are reachable within 3 taps from the Home screen. The usability testing SUS score of 76.5 (Good range) confirms that the design goals were achieved. The persistent emergency call banner on every screen ensures that professional help is always one tap away regardless of which screen the user is on.

7.2.3 Objective 3: Develop an AI Triage Module

The Google Gemini API (gemini-2.5-flash) was successfully integrated to provide dynamic, structured emergency guidance via three input modalities: text, voice (via expo-speech-recognition), and visual image input (via expo-image-picker and the Gemini Vision capability). The AI module returned clinically appropriate responses for

96% of 50 standardised test descriptions (43 Correct, 5 Partially Correct). Error handling ensures that API failures never leave the user without guidance, falling back to the offline scenario library automatically.

7.2.4 Objective 4: Implement an Improvised Resource Engine

The improvised resource recommendation engine is implemented as the `imp_resources` table in the SQLite database, with at least one and up to three household substitute mappings per scenario. Resource suggestions are accessible via an expandable accordion on every scenario screen, ensuring they are available without occupying primary screen space. The Gemini API system prompt also instructs the AI to include improvised resource alternatives in every online response, extending the engine to cover novel scenarios not in the pre-built library.

7.2.5 Objective 5: Evaluate Usability and Effectiveness

Usability evaluation was conducted through 15 functional test cases (all PASS) and a usability study with 15 representative participants yielding a SUS score of 76.5. The application met 6 of 8 non-functional requirements fully, with 2 partial results attributable to external API load conditions and complex multi-symptom edge cases rather than application deficiencies. The evaluation confirms that AidField is a functional, reliable, and usable emergency first aid tool.

7.3 Lessons Learnt

- Prompt engineering is an iterative discipline: achieving consistent, structured AI responses required numerous refinement cycles. The final system prompt, which specifies fixed section labels and prohibits markdown, was essential to reliable response parsing.
- Offline-first design must be a founding constraint, not an afterthought: making the offline tier primary from the start of the project prevented significant rework and ensured every feature had an offline fallback.
- Physical device testing is irreplaceable: the tab bar inset issue on the Infinix Note 30 Pro was not observable on the emulator and would have shipped as a bug without physical device testing.
- The Gemini API free tier has significant daily request limits: production deployment will require a paid tier to serve real users without disruption. Rate

limit handling and offline fallback are therefore not optional features but core requirements.

- Swahili translation of all 50 scenarios represented a substantial content effort. For future expansion of the scenario library, a translation workflow or community contribution model should be established.

7.4 Recommendations for Future Work

- Real speech recognition in Expo Go: The current implementation requires a custom development build for expo-speech-recognition. Future work should explore publishing AidField on the Google Play Store and Apple App Store, which will enable full voice functionality for all users without sideloading.
- Offline AI via on-device models: Explore integration of smaller, on-device language models (such as Gemma or Phi-3 Mini) to provide AI-quality guidance without internet connectivity, removing the dependency on the Gemini API for the most critical use cases. API key security hardening: Migrate the Gemini API key from build-time .env injection to a lightweight proxy backend (such as a Cloud Function or Firebase Function), ensuring the key cannot be extracted from the APK binary in a production release serving real users.
- User accounts and history: Implement optional Google Sign-In to allow users to save previous queries, track learning, and receive personalised scenario recommendations based on their location and history.
- Community scenario validation: Establish a review process involving Kenya Red Cross and clinical professionals to validate new scenarios contributed to the library, enabling the content to expand beyond the initial 50 scenarios.
- Privacy policy and terms of use: Add in-app privacy policy and terms of use screens before any public distribution, as required by Google Play and Apple App Store guidelines.
- Expansion to other East African contexts: The architecture of AidField is replicable. Future work should adapt the scenario library, emergency contacts, and contextual content for Uganda, Tanzania, and Rwanda.

7.5 Budget and Resource Summary

Table 7.1: Project Budget Summary

Item	Estimated Cost (KES)	Actual Cost (KES)
Development Laptop (existing hardware)	0	0
Google Gemini API (free tier during development)	0	0
Node.js, React Native, Expo, VS Code (open source)	0	0
Android Studio (free)	0	0
GitHub repository (free tier)	0	0
Physical test device — Infinix Note 30 Pro (existing)	0	0
Internet / mobile data for development and testing	1500	1000
Usability testing participant transport and refreshments	500	0
TOTAL	2000	1000

Note: The Google Gemini API was used on the free tier throughout the development and evaluation phases of this project at zero cost. The production release will require a paid API tier estimated at approximately 1,500 KES per month for a Kenya-scale deployment.

7.6 Conclusion

AidField successfully demonstrates that an offline-first, AI-enhanced emergency first aid application is technically feasible, usable, and impactful within the Kenyan context. The application delivers reliable guidance for 50 evidence-based emergency scenarios entirely without internet connectivity, and enhances that guidance with real-time AI triage, voice input, and visual analysis when connectivity is available. The dual-tier architecture ensures that the most vulnerable users — those in rural areas, during network outages, or in disaster zones — are never left without actionable emergency guidance.

The project contributes to three bodies of knowledge: it advances mHealth design for low-resource environments, demonstrates a practical pattern for offline-first AI application architecture, and provides a validated framework that can be replicated and

extended across East Africa and other emerging market contexts. It is the belief of the author that AidField, in its production form, has the potential to save lives.

REFERENCES

- American Red Cross. (2022). First Aid App. American Red Cross. <https://www.redcross.org/take-a-class/first-aid/first-aid-app>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. Agile Alliance. <https://agilemanifesto.org>
- Google. (2024). Gemini API documentation. Google AI for Developers. <https://ai.google.dev/docs>
- International Liaison Committee on Resuscitation. (2021). ILCOR consensus on science with treatment recommendations. *Circulation*, 144(16), e1–e29.
- Kenya Red Cross Society. (2019). Community-based first aid: Training manual for lay responders. Kenya Red Cross Society.
- Lockey, A. S., Lin, Y., & Cheng, A. (2021). Impact of CPR quality on clinical outcomes of cardiac arrest: A systematic review. *Resuscitation*, 165, 137–144. <https://doi.org/10.1016/j.resuscitation.2021.06.010>
- Statista. (2023). Number of smartphone users worldwide from 2016 to 2028. Statista. <https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/>
- St John Ambulance. (2023). First Aid app. St John Ambulance. <https://www.sja.org.uk/get-advice/first-aid-app/>
- Tintinalli, J. E., Ma, O. J., Yealy, D. M., Meckler, G. D., Stapczynski, J. S., Cline, D. M., & Thomas, S. H. (Eds.). (2020). *Tintinalli's emergency medicine: A comprehensive study guide* (9th ed.). McGraw-Hill Education.
- World Health Organization. (2021). Injuries and violence. WHO Fact Sheet. <https://www.who.int/news-room/fact-sheets/detail/injuries-and-violence>

APPENDIX A: USER MANUAL

A.1 Introduction

This appendix provides a concise user manual for AidField. It is intended for non-technical end users and covers installation, navigation, and the use of all key features.

A.2 Installation

A.2.1 Android Installation

16. Download the AidField APK file from the provided download link or QR code.
17. Open your Android device's Settings and navigate to Security or Privacy.
18. Enable "Install from Unknown Sources" or "Install Unknown Apps" for your browser or file manager.
19. Open the downloaded APK file and tap Install.
20. Once installed, open AidField. The app will initialise the offline database on first launch (this takes up to 2 seconds).

A.2.2 iOS Installation (*TestFlight*)

21. Download the TestFlight app from the Apple App Store if not already installed.
22. Open the invitation link provided by the developer in TestFlight.
23. Tap Accept and then Install to install AidField via TestFlight.
24. Open AidField. The offline database will initialise on first launch.

A.3 Using AidField

A.3.1 Home Screen (*Search Tab*)

The Home screen is your starting point in an emergency. It contains:

- Emergency Call card: Tap to call 999 or 112 immediately.
- Search bar: Type a description of the emergency (e.g. "severe bleeding", "choking adult"). If online, the Ask AI button appears — tap it for AI guidance. If offline, the button searches the local scenario library.
- Voice Search: Tap the green microphone button and describe the emergency aloud. The app will process your speech and retrieve guidance.

- Visual Input: Tap the purple camera button to photograph an injury. If online, the AI will analyse the image. If offline, the photo is saved to your gallery for later analysis.
- Critical Emergencies: Six quick-access cards at the bottom for the most common life-threatening emergencies. Tap any card to go directly to its guidance.

A.3.2 Scenarios Tab

The Scenarios tab shows all 11 emergency categories. Tap a category to see the scenarios within it. Tap a scenario to open the full step-by-step guidance.

A.3.3 Scenario Screen

Each scenario screen shows:

- Urgency badge: RED (Life-Threatening), AMBER (Urgent), or GREEN (Moderate).
- Read Aloud button: Tap to hear all steps read aloud. The active step is highlighted. Tap Stop to end.
- Language toggle: Switch between English and Kiswahili for the step instructions.
- Improvised Resources: Tap to expand and see household substitutes for standard medical supplies.

A.3.4 Settings Tab

In Settings you can:

- Change the display language between English and Kiswahili.
- Toggle Read Steps Aloud on or off — when on, steps begin reading automatically every time you open a scenario.
- Adjust the reading speed to Slow, Normal, or Fast.
- Access the Emergency Contacts screen with all Kenya emergency numbers.

A.3.5 Emergency Contacts

Access Emergency Contacts from the Settings tab or by tapping any emergency call banner. Tap any contact to dial. A confirmation dialog appears before any call is placed.

Key numbers: 999 (Police and Ambulance), 112 (Works without airtime or SIM), Kenya Red Cross 0800 723 999 (Free, 24 hours).

A.4 Offline and Online Features

Table A.1: Feature Availability by Connectivity

Feature	Offline	Online
Browse 50 emergency scenarios	Available	Available
Keyword search for scenarios	Available	Available
Read steps aloud (TTS)	Available	Available
Swahili language toggle	Available	Available
Emergency contacts dial	Available	Available
AI guidance (text query)	Not available	Available
Voice input to AI	Routes to offline library	Available with AI
Visual input AI analysis	Photo saved to gallery only	Available with AI

APPENDIX B: GEMINI API SYSTEM PROMPT

The following system prompt is used for all Gemini API requests in AidField. It is concatenated with the user message before being sent to the API.

You are AidField, an emergency first aid assistant for Kenya. Your role is to provide calm, clear, step-by-step first aid guidance to non-medical bystanders.

STRICT RULES:

- Never use asterisks, hashes, or any markdown symbols in your response.
- Keep each step to one short, clear sentence.
- Maximum 6 steps total.
- Always include calling 999 or 112.
- Always suggest a household substitute for at least one key medical supply.
- Be direct and calm — the user may be panicking.
- Never diagnose — only guide on immediate first aid actions.

You must respond in this exact format with no deviation:

URGENCY: [LIFE-THREATENING or URGENT or MODERATE]

IMMEDIATE ACTION: [One sentence on what to do right now]

STEPS:

1. [First step]
2. [Second step]
3. [Third step]

4. [Fourth step]

5. [Fifth step]

6. [Sixth step]

IMPROVISED RESOURCES:

- [Medical item]: [Household substitute]

CALL: [One sentence on when to call 999 or 112]